

Assignment II: Slicing, Arrays, Functions, and Rotations

Wan Zikri Hakim Nizam

[https://github.com/hakimwsu/wsu_math300/blob/b5af0cdb/assignments/Hakim.Assignment_2%20\(1\).pdf](https://github.com/hakimwsu/wsu_math300/blob/b5af0cdb/assignments/Hakim.Assignment_2%20(1).pdf)

September 27, 2026

Total: 100 points. Questions 1 to 3 (75 points: Q1 is 20 points, Q2 and Q3 are 25 points each, including the 5-point Submission), plus Question 4 (25 bonus points).

This assignment pulls together everything from Sep 9 to Sep 18: list and NumPy array slicing, array arithmetic, writing your own functions, generating lists, and building rotation matrices. **All arrays and lists in this assignment are 1D or 2D only.** You will not need (and should not use) any 3D or higher-dimensional arrays anywhere below.

- Each of Questions 1 to 3 is split into **Part (a)** and **Part (b)**.
- **Question 4 is an entirely bonus question worth 25 points.** It asks you to explore the relationship between $SU(2)$ and $SO(3)$, the famous “double cover,” using only the tools you already have (complex numbers, 2D arrays, and functions).
- Write your code in the cell provided for each part, and make sure it **prints** your final answer(s) clearly, ideally with an f-string that says what the printed value represents.
- **Submission (5 points):** see the instructions at the very end of this document.

AI Usage Policy

No AI tools (ChatGPT, Claude, GitHub Copilot’s suggestion/chat features, etc.) may be used to generate, complete, or rewrite your solutions for this assignment. All code and explanations must be your own work.

Exception and disclosure requirement: If you have **GitHub Copilot active** in your editor (e.g., inline autocomplete suggestions appearing as you type), you do not need to disable it, but you **must disclose this** by adding a short section stating:

- That Copilot (or any other AI tool) was active while you worked, and
- Briefly, how you used or did not use its suggestions (e.g., “Copilot was active but I did not accept any of its suggestions for graded cells” or “I accepted a Copilot suggestion for the boilerplate for loop in Q3(a) but wrote the logic myself”).

AI Usage Disclosure (student to fill in):

(If GitHub Copilot or any other AI tool was active while you worked on this assignment, describe that here. If no AI tool was active at all, write “No AI tools were used.”)

Setup

```
1 import numpy as np
```

Question 1: Slicing, Lists and NumPy Arrays (20 points)

(a) [12 pts] **Lists.** Create a list of at least 8 numbers called `my_list`. Using slicing (no loops), print:

1. The first 4 elements.
2. The last 4 elements.
3. Every 3rd element, starting from the first.
4. The list reversed.

```
1 # Code for Q1(a)
2
3 my_list = [12, 25, 37, 48, 51, 63, 74, 89, 90, 102]
4
5 print("--- Q1(a): Lists ---")
6 print(f"1. First 4 elements: {my_list[:4]}")
7 print(f"2. Last 4 elements: {my_list[-4:]}")
8 print(f"3. Every 3rd element from first: {my_list[::3]}")
9 print(f"4. List reversed: {my_list[::-1]}")
```

Output:

```
--- Q1(a): Lists ---
1. First 4 elements: [12, 25, 37, 48]
2. Last 4 elements: [74, 89, 90, 102]
3. Every 3rd element from first: [12, 48, 74, 102]
4. List reversed: [102, 90, 89, 74, 63, 51, 48, 37, 25, 12]
```

(b) [8 pts] **2D arrays.** Create a 5×5 NumPy array `M` containing the integers 1 through 25 (using `np.arange` and `.reshape`). Using slicing, print:

1. The 3rd row of `M`.
2. The 2nd column of `M`.
3. The 2x2 sub-matrix in the *bottom-right* corner of `M`.
4. `M` with its rows reversed, using slicing, **not** `np.flip`.

```
1 # Code for Q1(b)
2
3 M = np.arange(1, 26).reshape(5, 5)
4
5 print("\n--- Q1(b): 2D NumPy Arrays ---")
6 print(f"Full Array M:\n{M}\n")
7 print(f"1. 3rd row of M: {M[2, :]}")
8 print(f"2. 2nd column of M: {M[:, 1]}")
9 print(f"3. 2x2 sub-matrix (bottom-right):\n{M[-2:, -2:]}")
10 print(f"4. M with rows reversed:\n{M[::-1, :]}")
```

Output:

```
--- Q1(b): 2D NumPy Arrays ---
Full Array M:
[[ 1  2  3  4  5]
 [ 6  7  8  9 10]
 [11 12 13 14 15]
 [16 17 18 19 20]
 [21 22 23 24 25]]

1. 3rd row of M: [11 12 13 14 15]
2. 2nd column of M: [ 2  7 12 17 22]
3. 2x2 sub-matrix (bottom-right):
[[19 20]
 [24 25]]
4. M with rows reversed:
[[21 22 23 24 25]
 [16 17 18 19 20]
 [11 12 13 14 15]
 [ 6  7  8  9 10]
 [ 1  2  3  4  5]]
```

Question 2: Arithmetic with NumPy Arrays (25 points)

(a) [15 pts] **Elementwise arithmetic with floats.** Create `x = np.linspace(1, 10, 10)` (this always gives you floats). Print `x + 3`, `x - 2`, `x * 4`, `x / 5`, `x ** 3`, and `np.sqrt(x)`. Then compute `y = np.sqrt(x) ** 2` and use `np.isclose(x, y)` to confirm that `y` matches `x` element by element, even though squaring a square root does not give back the *exact* original floating-point value.

Example of how `np.isclose` works: `np.isclose(0.0000001, 0)` returns `True`, because the two values are equal up to a small numerical tolerance, even though `0.0000001 == 0` would return `False`. This is exactly why we use `np.isclose` instead of `==` when comparing floating-point results, such as your `x` and `y` above: after the square root and the squaring, the values will differ from the originals by something like `1e-15`, not exactly `0.0`.

```
1 # Code for Q2(a) part 1
2
3 x = np.linspace(1, 10, 10)
4
5 print("--- Q2(a): Elementwise Operations ---")
6 print(f"x + 3 = {x + 3}")
7 print(f"x - 2 = {x - 2}")
8 print(f"x * 4 = {x * 4}")
9 print(f"x / 5 = {x / 5}")
10 print(f"x ** 3 = {x ** 3}")
11 print(f"np.sqrt(x) = {np.sqrt(x)}")
```

Output:

```
--- Q2(a): Elementwise Operations ---
x + 3 = [ 4.  5.  6.  7.  8.  9. 10. 11. 12. 13.]
x - 2 = [-1.  0.  1.  2.  3.  4.  5.  6.  7.  8.]
x * 4 = [ 4.  8. 12. 16. 20. 24. 28. 32. 36. 40.]
```

```

x / 5 = [0.2 0.4 0.6 0.8 1. 1.2 1.4 1.6 1.8 2. ]
x ** 3 = [ 1. 8. 27. 64. 125. 216. 343. 512. 729. 1000.]
np.sqrt(x) = [1. 1.41421356 1.73205081 2. 2.23606798
2.44948974
2.64575131 2.82842712 3. 3.16227766]

```

```

1 # Code for Q2(a) part 2
2
3 y = np.sqrt(x) ** 2
4 matches_elementwise = np.isclose(x, y)
5 print(f"\nDoes y match x elementwise? {matches_elementwise}")
6 print(f"All elements match within tolerance: {np.all(matches_elementwise)}")

```

Output:

```

Does y match x elementwise? [ True  True  True  True  True  True  True  True
 True  True  True]
All elements match within tolerance: True

```

(b) [10 pts] **Matrix multiplication and standardization.** Create two 3×3 matrices A and B of your choice (any numbers). Compute their matrix product $A @ B$ (**not** elementwise multiplication) and print it. Then confirm, using `np.allclose(A @ B, B @ A)`, that matrix multiplication is generally **not commutative** (this should print `False`, unless you happened to pick special matrices). Finally, compute a “standardized” version of A: subtract `A.mean()` and divide by `A.std()`. Print the standardized array, and confirm its new mean is (approximately) 0 using `np.isclose`.

```

1 # Code for Q2(b)
2
3 A = np.array([[2, 5, 8], [3, 1, 7], [4, 9, 6]])
4 B = np.array([[1, 0, 4], [2, 3, 5], [9, 1, 2]])
5
6 AB = A @ B
7 BA = B @ A
8
9 print("\n--- Q2(b): Matrix Multiplication & Standardization ---")
10 print(f"A @ B =\n{AB}")
11 is_commutative = np.allclose(AB, BA)
12 print(f"Is matrix multiplication commutative (A @ B == B @ A)? {is_commutative}")
13
14 A_std = (A - A.mean()) / A.std()
15 print(f"\nStandardized Array A:\n{A_std}")
16 mean_is_zero = np.isclose(A_std.mean(), 0)
17 print(f"Is standardized mean close to 0? {mean_is_zero} (Actual mean: {A_std.mean():.2e})")

```

Output:

```

--- Q2(b): Matrix Multiplication & Standardization ---
A @ B =
[[84 23 49]
 [68 10 31]
 [76 33 73]]
Is matrix multiplication commutative (A @ B == B @ A)? False

```

```
Standardized Array A:
[[-1.161895      0.          1.161895   ]
 [-0.77459667  -1.54919334   0.77459667]
 [-0.38729833   1.54919334   0.38729833]]
Is standardized mean close to 0? True (Actual mean: 0.00e+00)
```

Question 3: Defining Functions and Generating Lists (25 points)

Background: the double factorial. For a positive integer n , the **double factorial** $n!!$ is the product of every second integer counting down from n to either 1 or 2 (whichever has the same parity as n). For example:

- $6!! = 6 \times 4 \times 2 = 48$
- $7!! = 7 \times 5 \times 3 \times 1 = 105$

By convention, $0!! = 1$ and $1!! = 1$.

(a) [15 pts] A simple generator function. Write a function `double_factorial(n)` that computes and returns $n!!$ for a single non-negative integer n , using a `for` (or `while`) loop. Then write a function `double_factorials_up_to(n)` that uses a loop over $k = 1, \dots, n$ (calling your `double_factorial` function each time) to build and return a list of $[1!!, 2!!, 3!!, \dots, n!!]$. Call it with $n = 10$ and print the result.

```
1 # Code for Q3(a)
2
3 def double_factorial(n):
4     """Computes n!! for a non-negative integer n."""
5     if n == 0 or n == 1:
6         return 1
7     result = 1
8     for k in range(n, 0, -2):
9         result *= k
10    return result
11
12 def double_factorials_up_to(n):
13     """Returns a list of [1!!, 2!!, ..., n!!]."""
14     return [double_factorial(k) for k in range(1, n + 1)]
15
16 print("--- Q3(a): Double Factorials ---")
17 df_list = double_factorials_up_to(10)
18 print(f"Double factorials up to 10!!: {df_list}")
```

Output:

```
--- Q3(a): Double Factorials ---
Double factorials up to 10!!: [1, 2, 3, 8, 15, 48, 105, 384, 945, 3840]
```

(b) [10 pts] A function with a custom recurrence. Write a function `custom_sequence(a0, a1, n)` that returns a list of the first n terms of the sequence defined by $s_0 = a_0$, $s_1 = a_1$, and $s_k = 2s_{k-1} + s_{k-2}$ for $k \geq 2$ (a “Pell-like” sequence). Give `a1` a default value of 1. Call your function with `a0 = 0`, using the default for `a1`, for $n = 10$ terms, and print the result.

```

1 # Code for Q3(b)
2
3 def custom_sequence(a0, a1=1, n=10):
4     """Returns the first n terms of s_k = 2*s_{k-1} + s_{k-2}."""
5     if n <= 0:
6         return []
7     if n == 1:
8         return [a0]
9
10    seq = [a0, a1]
11    for k in range(2, n):
12        next_term = 2 * seq[k - 1] + seq[k - 2]
13        seq.append(next_term)
14    return seq
15
16 print("\n--- Q3(b): Custom Recurrence Sequence ---")
17 pell_seq = custom_sequence(a0=0, a1=1, n=10)
18 print(f"First 10 terms of sequence (a0=0, a1=1): {pell_seq}")

```

Output:

```

--- Q3(b): Custom Recurrence Sequence ---
First 10 terms of sequence (a0=0, a1=1): [0, 1, 2, 5, 12, 29, 70, 169,
408, 985]

```

Question 4 (Bonus, 25 points): $SU(2)$ and the Double Cover of $SO(3)$

You've already built rotation matrices $R_x, R_y, R_z \in SO(3)$: real 3×3 matrices that rotate vectors in ordinary 3D space. There is a second way to describe the *same* rotations, using 2×2 **complex** matrices instead. This is the group $SU(2)$.

Reminder: $R_z(\theta)$. For reference, here again is the rotation-about- z matrix:

$$R_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Background (given, no derivation needed). A rotation by angle θ about a unit-vector axis $n = [n_x, n_y, n_z]$ (given to you as a plain Python **list** of three numbers), viewed as an element of $SU(2)$, is given by:

$$U(\theta, n) = \begin{bmatrix} \cos\left(\frac{\theta}{2}\right) - i n_z \sin\left(\frac{\theta}{2}\right) & -(n_y + i n_x) \sin\left(\frac{\theta}{2}\right) \\ (n_y - i n_x) \sin\left(\frac{\theta}{2}\right) & \cos\left(\frac{\theta}{2}\right) + i n_z \sin\left(\frac{\theta}{2}\right) \end{bmatrix}$$

You are also given the three Pauli matrices. In math notation, they are:

$$\sigma_x = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad \sigma_y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}, \quad \sigma_z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

Every $U \in SU(2)$ maps to an actual 3×3 rotation matrix $R(U) \in SO(3)$ via the formula:

$$R(U)_{jk} = \frac{1}{2} \text{tr}(\sigma_j U \sigma_k U^\dagger), \quad j, k \in \{x, y, z\}$$

where $U^\dagger$ is the conjugate transpose of U (`U.conj().T` in NumPy). Recall that the **trace** of a square matrix, written $\text{tr}(\cdot)$, is just the sum of its diagonal entries; in NumPy you can compute it directly with `np.trace(A)`.

Note: how R_{jk} connects to $R_z(\theta)$. The nine numbers R_{jk} for $j, k \in \{x, y, z\}$ are just the nine entries of the 3×3 matrix, with j giving the row and k giving the column. So checking that `su2_to_so3(U(theta, n))` equals `Rz(theta)` just means all nine of these entries should match. When $n = [0, 0, 1]$, the formula for $U(\theta, n)$ becomes a diagonal matrix, so U commutes with σ_z , giving $R_{zz} = 1$ and $R_{xz} = R_{yz} = R_{zx} = R_{zy} = 0$. For the remaining entries, using σ_x and σ_y conjugated by U and the identity $\text{tr}(\sigma_j \sigma_k) = 2$ if $j = k$, else 0, gives $R_{xx} = R_{yy} = \cos(\theta)$, $R_{yx} = \sin(\theta)$, and $R_{xy} = -\sin(\theta)$. Arranging these nine values into the 3×3 grid reproduces exactly $R_z(\theta)$.

```

1 # Setup code for Q4
2
3 def Rz(theta):
4     """Standard 3D rotation matrix about the z-axis."""
5     c = np.cos(theta)
6     s = np.sin(theta)
7     return np.array([[c, -s, 0], [s, c, 0], [0, 0, 1]])
8
9 # Pauli matrices
10 sigma_x = np.array([[0, 1], [1, 0]], dtype=complex)
11 sigma_y = np.array([[0, -1j], [1j, 0]], dtype=complex)
12 sigma_z = np.array([[1, 0], [0, -1]], dtype=complex)
13 paulis = [sigma_x, sigma_y, sigma_z]

```

(a) [15 pts] Write a function `U(theta, n)` that takes an angle `theta` and an axis `n = [nx, ny, nz]` (a Python list), and returns the 2×2 complex NumPy array given by the formula above. Write a function `su2_to_so3(M)` that builds the 3×3 real matrix $R(M)$ using the second formula above. Using `n = [0, 0, 1]` (the z -axis) and an angle `theta` of your choice, confirm with `np.allclose` that `su2_to_so3(U(theta, n))` matches `Rz(theta)`.

```

1 # Code for Q4(a)
2
3 def U(theta, n):
4     """Returns 2x2 SU(2) matrix for rotation by angle theta about axis n=[
5     nx, ny, nz]."""
6     nx, ny, nz = n
7     half_th = theta / 2.0
8     cos_h = np.cos(half_th)
9     sin_h = np.sin(half_th)
10
11     u11 = cos_h - 1j * nz * sin_h
12     u12 = -(ny + 1j * nx) * sin_h
13     u21 = (ny - 1j * nx) * sin_h
14     u22 = cos_h + 1j * nz * sin_h
15
16     return np.array([[u11, u12], [u21, u22]], dtype=complex)
17
18 def su2_to_so3(M):
19     """Maps a 2x2 SU(2) matrix M to a 3x3 SO(3) rotation matrix R."""
20     R = np.zeros((3, 3))
21     M_dag = M.conj().T

```

```

21     for j in range(3):
22         for k in range(3):
23             val = 0.5 * np.trace(paulis[j] @ M @ paulis[k] @ M_dag)
24             R[j, k] = val.real
25     return R
26
27 print("--- Q4(a): SU(2) to SO(3) Mapping ---")
28 test_angle = np.pi / 4
29 z_axis = [0, 0, 1]
30
31 U_z = U(test_angle, z_axis)
32 R_mapped = su2_to_so3(U_z)
33 R_direct = Rz(test_angle)
34
35 print(f"Mapped R(U) Matrix:\n{R_mapped}")
36 print(f"Matches direct Rz(theta)? {np.allclose(R_mapped, R_direct)}")

```

Output:

```

--- Q4(a): SU(2) to SO(3) Mapping ---
Mapped R(U) Matrix:
[[ 0.70710678 -0.70710678  0.         ]
 [ 0.70710678  0.70710678  0.         ]
 [ 0.         0.         1.         ]]
Matches direct Rz(theta)? True

```

(b) [10 pts] Here is a simpler way to see the double cover, using R_z and rotating **twice**. Let $n = [0, 0, 1]$ and $\theta = \text{np.pi}$ (a half turn). Compute $U_{\text{half}} = U(\theta, n)$, then “rotate twice” by composing this matrix with itself: $U_{\text{full}} = U_{\text{half}} @ U_{\text{half}}$.

1. Use `np.allclose` to show that U_{full} equals $-\text{np.eye}(2)$ (the *negative* identity), not the identity $\text{np.eye}(2)$.
2. Use `su2_to_so3` to show that `su2_to_so3(U_full)` matches `Rz(2 * np.pi)`, which should simply be the identity 3×3 matrix.
3. Print a one or two sentence explanation of what this shows.

```

1 # Code for Q4(b)
2
3 print("\n--- Q4(b): Double Cover Verification ---")
4 theta_half = np.pi
5 U_half = U(theta_half, z_axis)
6 U_full = U_half @ U_half
7
8 # 1. Compare U_full to -I
9 is_neg_I = np.allclose(U_full, -np.eye(2))
10 print(f"1. Is U_full equal to -I (2x2)? {is_neg_I}")
11
12 # 2. Compare mapped matrix to Rz(2*pi)
13 R_full_mapped = su2_to_so3(U_full)
14 R_2pi = Rz(2 * np.pi)
15 is_SO3_identity = np.allclose(R_full_mapped, R_2pi)
16 print(f"2. Does su2_to_so3(U_full) match Rz(2*pi)? {is_SO3_identity}")

```

```

17
18 # 3. Explanation
19 explanation = (
20     "A full  $2\pi$  rotation in SU(2) evaluates to -I rather than the
      identity matrix I. "
21     "However, both I and -I in SU(2) map to the exact same 3x3 identity
      rotation matrix in SO(3). "
22     "Because every 3D rotation in SO(3) corresponds to two distinct
      matrices (+U and -U) in SU(2), "
23     "SU(2) acts as a double cover of SO(3). "
24 )
25 print(f"\n3. Explanation:\n{explanation}")

```

Output:

```

--- Q4(b): Double Cover Verification ---
1. Is U_full equal to -I (2x2)? True
2. Does su2_to_so3(U_full) match Rz(2*pi)? True

3. Explanation:
A full  $2\pi$  rotation in SU(2) evaluates to -I rather than the identity
matrix I. However, both I and -I in SU(2) map to the exact same 3x3
identity rotation matrix in SO(3). Because every 3D rotation in SO(3)
corresponds to two distinct matrices (+U and -U) in SU(2), SU(2) acts
as a double cover of SO(3).

```

Submission Instructions (5 points)

Submit this assignment in two parts:

1. **CrowdMark:** Submit your completed notebook (or a PDF/printout of it) through CrowdMark as normal.
2. **GitHub (worth 5 points):** Push this completed notebook to a GitHub repository for Assignment II, and include the link to your GitHub repository as part of your CrowdMark submission.
 - **Either** make the repository **public**, so the link alone is enough, **or**
 - keep it **private** and add the instructor as a collaborator using the GitHub username say-yas.
3. Double check that the link you submit actually opens your repository before submitting.